

Alcohol Label Verification

Design Document — approach, implementation, and architecture decisions

Live demo: <https://alcohol-label-verification-fawn.vercel.app> · Repository:

github.com/aaryannaz/alcohol-label-verification

1. Summary

TTB reviewers process about 150,000 alcohol-label applications a year, and much of the work is rote matching: confirming the brand name, alcohol content, and government warning on the label match the approved application. This app automates that comparison. The reviewer uploads the approved COLA application form and the label artwork; Gemini Vision reads each into structured fields; a pure, rule-based engine compares them field-by-field and runs the TTB compliance checks. It returns in about 2 seconds per label, handles beer, wine, and distilled spirits, and offers a batch mode that verifies many labels in parallel. It is a standalone prototype deployed to a public URL, intentionally a single, simple deployable unit.

```
Upload (COLA form + label) -> Gemini Vision: structured field JSON
    -> Pure rule engine: field-by-field comparison + TTB compliance checks
    -> Pass / mismatch verdict per field, plus label-only advisory checks
```

2. What the customers said they wanted

Drawn from the discovery interviews in the project brief:

- **Speed above all.** A prior scanning-vendor pilot took 30-40 seconds per label and was abandoned: *"if we can't get results back in about 5 seconds, nobody's going to use it"* (Sarah Chen, Deputy Director).
- **The core task is matching.** Agents mostly confirm the value on the label equals the value on the application - brand name, ABV, government warning - which is exactly what should be automated.
- **Judgment, not brittle matching.** *"STONE'S THROW"* on the label vs *"Stone's Throw"* in the application is obviously the same product (Dave Morrison, 28-year agent). Cosmetic differences must not be flagged as mismatches.
- **The government warning must be exact.** Word-for-word, with the *"GOVERNMENT WARNING:"* heading in all caps; a title-case heading is a real rejection reason (Jenny Park caught one).
- **A dead-simple interface** for a workforce that is half over 50 and varies widely in tech comfort: *"something my 73-year-old mother could figure out... clean, obvious, no hunting for buttons"* (Sarah).
- **Batch uploads.** Importers dump 200-300 applications at once during peak season, and today they are processed one at a time.
- **Three beverage types** (beer, wine, distilled spirits) with type-specific required fields; a standalone proof-of-concept, not integrated with the live COLA system, storing nothing sensitive (Marcus Williams, IT).

3. How we implemented it

- **Two-stage pipeline.** Gemini extraction (image -> JSON) is decoupled from a pure, rule-based validation step. Because validation has no I/O, re-checking after the reviewer edits a field costs no additional AI call, and it is deterministic and fully unit-tested.
- **Speed.** gemini-2.5-flash runs with its "thinking" phase disabled and returns in about 2s; an eval harness measures p50/p95 latency to keep it under the 5-second bar.

- **Lenient-but-correct matching.** A normalize step ignores case, punctuation, and common abbreviations, so "STONE'S THROW" matches "Stone's Throw" - exactly Dave's example - without passing genuine mismatches.
- **Government warning.** Checked against the exact statutory text; the heading must be all caps, and a title-case heading returns a specific FAIL_HEADING_FORMAT verdict - the rejection Jenny described.
- **Simple UI.** One clean editable column, an Instructions page, a red asterisk on required fields, and explicit empty / loading / error states - no hunting for buttons.
- **Batch mode.** Files are verified in parallel and each row lands on an at-a-glance Pass / Needs-attention verdict; a separate front and back panel can be paired into one review item.
- **Beverage-aware.** The field set and the required / conditional / optional rules switch per product category, all driven from a single source of truth so the schema, validation, API, and UI never drift.
- **Quality is measured, not assumed.** An eval harness renders ground-truth labels, runs the real extraction path, and scores each field (~98-99%); prompt and model changes are gated on it.

What the tool validates. Coverage is scoped to the selected beverage type, so a beer is never asked for wine or spirits fields:

- **Common (all types):** brand name, class/type designation, alcohol content, net contents, name/address of the bottler or importer, country of origin for imports, and the government warning.
- **Conditional disclosures:** FD&C Yellow #5, cochineal/carmine, aspartame (with an all-caps format check), and sulfites - surfaced when present for the reviewer to confirm applicability.
- **Wine:** grape varietal, the appellation-of-origin trigger, vintage date, and a table-wine alcohol-content exemption.
- **Distilled spirits:** statement of age, commodity statement, coloring materials, wood treatment, and state of distillation.
- **Label-only advisory checks:** net-contents unit system, standard of fill, and origin coherence.

4. Why this architecture

- **AI model - Google Gemini Vision (gemini-2.5-flash, thinking disabled).** The task is to read text off a photograph and return structured data, so we need a vision model with enforced JSON-schema output - Gemini provides both. Reading a label is perception, not reasoning, so disabling the model's "thinking" phase cut latency from ~7s to ~2s with no measured accuracy loss - the single change that meets the 5-second bar. We chose 2.5-flash over flash-lite (more accurate on judgment fields, e.g. telling a fanciful name from a class/type) and over flash-with-thinking (too slow).
- **Backend - FastAPI (Python), not Flask.** Async request handling lets a batch fan out concurrent Gemini calls; Pydantic gives typed request validation and the field schema for free; OpenAPI/Swagger docs are automatic; and it serves the static UI itself, so the whole app is one deployable unit. Flask would need bolt-on extensions for each of those.
- **Frontend - vanilla HTML/CSS/JS, not Next.js/React.** The UI is a few forms and a results table, not an application that needs a single-page framework. No build step, no node_modules, no hydration - just static files served by the same backend. Fewer moving parts directly serves the "clean, obvious" mandate and keeps the deploy a single artifact.
- **Hosting - Vercel.** Zero-config serverless Python, deploy-on-git-push, and a generous free tier - ideal for a prototype that needs a public URL reviewers can test, with fast iteration.

Honest trade-offs (prototype vs production). (a) IT noted TTB's network blocks outbound traffic to many ML endpoints; this prototype calls the hosted Gemini API, so a production deployment would allowlist it or

run an Azure-hosted / on-prem model. (b) The rate limiter is in-memory and single-instance; behind multiple instances it would move to a shared store such as Redis. (c) Nothing is persisted, matching "*we're not storing anything sensitive.*" (d) Typography rules - bold, minimum type size, placement - are not verifiable from extracted text and are left to the reviewer's eye.

Validated, not asserted. The decisions above are backed by evidence rather than claims:

- Extraction accuracy is about 98-99% on a labeled eval set; the same harness reports p50/p95 latency and flags anything over the 5-second bar.
- 81 automated tests (unit + API) plus ruff linting run in CI on every push; the rule-based validation engine - the core compliance logic - is the most heavily tested module.
- Uploads are screened three ways (file extension, content-type, and magic-byte signature) with a 10 MB cap; the API is rate-limited per IP and sends standard security headers.
- The prototype is deployed to a public URL and has been verified end-to-end against a live label.